

API Gateway Routing, Filtering & Request Policies

EPIC 07 - Advanced API Architecture, Observability & Production System Design

JIRA Task 01 | Date: 05-Oct-2026 | Effort: 8 hours | Priority: High

Prepared by: Shivasai

1. Objective

Learn why an API Gateway is required in a microservices architecture and implement routing between multiple services. Instead of calling every service on its own port, the client calls **one entry point** (the Gateway on port 8080). The Gateway decides which service should receive each request.

2. Why a Gateway?

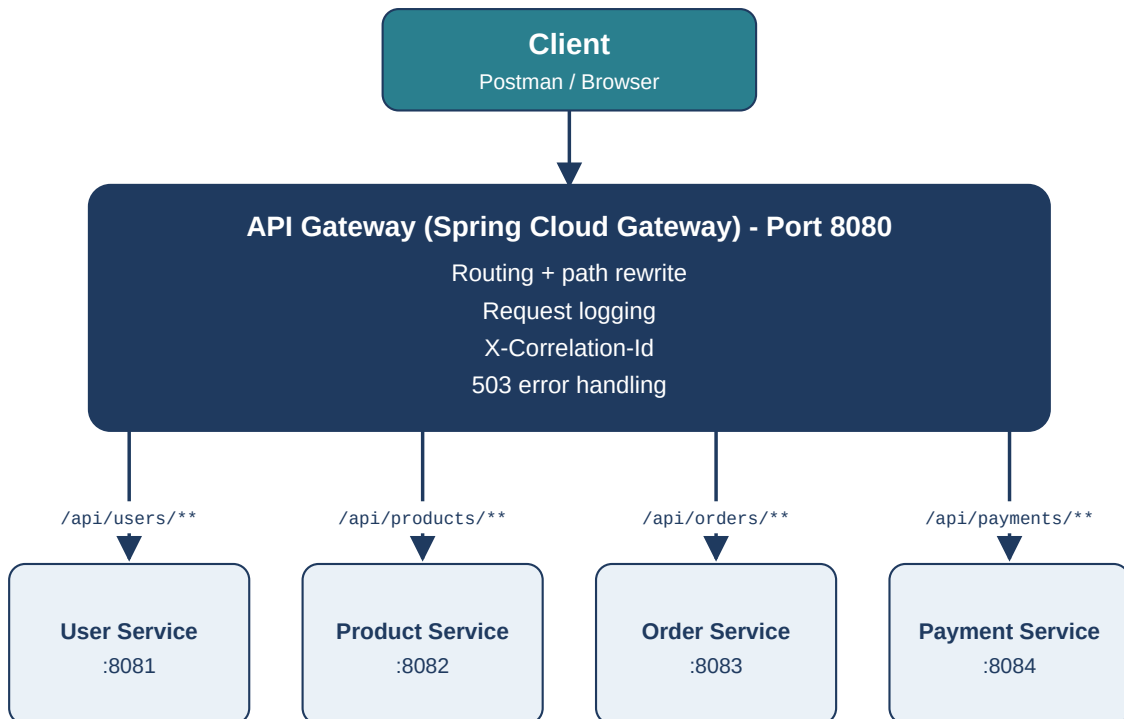
Without a Gateway	With a Gateway
Client must know every service address and port	Client knows only one address: localhost:8080
Logging, security and error handling repeated in each service	Cross-cutting features are added once, in one place
No single ID to follow a request across services	Every request gets a correlation ID at the entry point
A stopped service gives the client a confusing error	Gateway returns a clean, controlled error message

3. Existing Services Reviewed (Step 2)

Service	Port	Path inside the service	Gateway path
user-service	8081	/users	/api/users/**
product-service	8082	/api/v1/products	/api/products/**
order-service	8083	/api/v1/orders	/api/orders/**
payment-service	8084	/api/v1/payments	/api/payments/**

The user-service uses /users while the others use /api/v1/. . . . The Gateway hides this difference by rewriting the path, so clients always use the same clean style: /api/<name>.

4. Architecture Diagram (Step 10)



Flow: Client -> API Gateway -> matching service. The Gateway adds the X-Correlation-Id header, logs the request and handles errors.

5. Implementation Summary

Step	Task	Status
1	Understand API Gateway (what, why, client-to-gateway, gateway-to-service)	Done
2	Review existing services (ports and paths)	Done
3	Create Spring Cloud Gateway project (api-gateway)	Done
4	Configure service routes	Done
5	Test each route using Postman	Done
6	Add Gateway request logging	Done
7	Add correlation / request ID	Done
8	Forward correlation ID to downstream services	Done
9	Test service-unavailable scenario	Done
10	Create architecture diagram	Done

6. Project Setup (Step 3)

Item	Value
Project name	api-gateway (folder: microservices/api-gateway)
Build tool	Maven
Main dependency	Spring Cloud Gateway (WebFlux / reactive)
Extra dependency	Spring Boot Actuator
Port	8080

Issue faced and fixed: the first routes returned 404 because the MVC version of the Gateway starter was selected, which reads routes from different settings. Switching to the WebFlux starter (spring-cloud-starter-gateway-server-webflux) fixed it.

7. Route Configuration (Step 4)

File: src/main/resources/application.yml

```
server:
  port: 8080

spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      server:
        webflux:
          routes:
            - id: user-service
              uri: http://localhost:8081
              predicates:
                - Path=/api/users/**
              filters:
                - RewritePath=/api/users(?<segment>/?.*), /users$\{segment}

            - id: product-service
              uri: http://localhost:8082
              predicates:
                - Path=/api/products/**
              filters:
                - RewritePath=/api/products(?<segment>/?.*), /api/v1/products$\{segment}

            - id: order-service
              uri: http://localhost:8083
              predicates:
                - Path=/api/orders/**
              filters:
                - RewritePath=/api/orders(?<segment>/?.*), /api/v1/orders$\{segment}

            - id: payment-service
              uri: http://localhost:8084
              predicates:
                - Path=/api/payments/**
              filters:
                - RewritePath=/api/payments(?<segment>/?.*), /api/v1/payments$\{segment}
```

Keyword	Meaning
id	Name of the route
uri	Address of the real service

Keyword	Meaning
predicates	Which incoming URL matches this route
filters (RewritePath)	Changes the path before forwarding, e.g. /api/users/5 becomes /users/5

8. Gateway Filter, Logging and Correlation ID (Steps 6-8)

A **global filter** runs for every request that passes through the Gateway. It does three jobs:

Job	How
Logging (Step 6)	Logs Request ID, HTTP method, URL, service name, response status and response time in ms
Correlation ID (Step 7)	Uses the incoming X-Correlation-Id if the client sent one, otherwise generates a UUID
Forwarding (Step 8)	Adds X-Correlation-Id to the request sent to the service and also to the response sent back

File: filter/RequestLoggingFilter.java

```
@Component
public class RequestLoggingFilter implements GlobalFilter, Ordered {

    private static final Logger log =
        LoggerFactory.getLogger(RequestLoggingFilter.class);
    private static final String HEADER = "X-Correlation-Id";

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        long start = System.currentTimeMillis();

        // Step 7: reuse incoming ID or create a new one
        String incoming = exchange.getRequest().getHeaders().getFirst(HEADER);
        final String requestId = (incoming == null || incoming.isBlank())
            ? UUID.randomUUID().toString() : incoming;

        String method = exchange.getRequest().getMethod().name();
        String url = exchange.getRequest().getURI().toString();

        // Step 8: forward the ID to the downstream service
        ServerHttpRequest request = exchange.getRequest().mutate()
            .header(HEADER, requestId).build();
        ServerWebExchange newExchange = exchange.mutate().request(request).build();
        newExchange.getResponse().getHeaders().add(HEADER, requestId);

        return chain.filter(newExchange).then(Mono.fromRunnable(() -> {
            Route route = newExchange.getAttribute(
                ServerWebExchangeUtils.GATEWAY_ROUTE_ATTR);
            String service = (route != null) ? route.getId() : "unknown";
            long time = System.currentTimeMillis() - start;
            log.info("RequestID={}" | Method={} | URL={} | Service={} | Status={} | Time={}ms",
                requestId, method, url, service,
                newExchange.getResponse().getStatusCode(), time);
        })));
    }

    @Override
    public int getOrder() { return -1; }
}
```

Sample log output:

```
RequestID=8805fe58-e783-48e2-938c-941ae51c1478 | Method=GET |
URL=http://localhost:8080/api/users | Service=user-service |
Status=200 OK | Time=1371ms
```

9. Error Handling (Step 9)

When a service is stopped, a raw connection error would confuse the client. A custom handler turns it into a clean JSON response with status **503 Service Unavailable**. Unknown paths return a clear 404 message.

File: `filter/GatewayErrorHandler.java`

```
@Component
@Order(-2)
public class GatewayErrorHandler implements WebExceptionHandler {

    @Override
    public Mono<Void> handle(ServerWebExchange exchange, Throwable ex) {
        var response = exchange.getResponse();
        if (response.isCommitted()) return Mono.error(ex);

        HttpStatusCode status = HttpStatusCode.valueOf(503);
        String message = "Service is temporarily unavailable. Please try again later.";

        if (ex instanceof ResponseStatusException rse
            && rse.getStatusCode().value() == 404) {
            status = rse.getStatusCode();
            message = "No route found for this request.";
        }

        log.error("Gateway error for {} : {}", exchange.getRequest().getURI(),
            ex.getMessage());
        response.setStatusCode(status);
        response.getHeaders().setContentType(MediaType.APPLICATION_JSON);
        String json = "{\"status\":\"" + status.value()
            + "\",\"error\":\"" + message + "\""
            + "\",\"path\":\"" + exchange.getRequest().getPath() + "\"}";
        return response.writeWith(Mono.just(
            response.bufferFactory().wrap(json.getBytes(StandardCharsets.UTF_8))));
    }
}
```

Response when user-service is stopped:

```
{"status":503,
 "error":"Service is temporarily unavailable. Please try again later.",
 "path":"/api/users"}
```

10. Testing with Postman (Step 5 and Step 9)

Request (via Gateway)	Result	Meaning
GET localhost:8080/api/users	200 OK	Routing works; users returned from user-service
GET localhost:8080/api/products	401 Unauthorized	Reached product-service; it requires a JWT token
GET localhost:8080/api/orders	405 Method Not Allowed	Reached order-service at /api/v1/orders (path rewrite works)
GET localhost:8080/api/payments	405 Method Not Allowed	Reached payment-service at /api/v1/payments (path rewrite works)
GET /api/users with user-service stopped	503 Service Unavailable	Gateway returned a controlled JSON error
Response header check	X-Correlation-Id present	Correlation ID generated and returned

A 401 or 405 is a valid proof of routing: the request passed through the Gateway and was answered by the correct service. Only the request type or the login token was not what that service expected.

11. Deliverables Checklist

Deliverable	Where / Evidence	Status
Spring Cloud Gateway	microservices/api-gateway project	Done
Route configuration	application.yml (section 7)	Done
Gateway filter	RequestLoggingFilter.java (section 8)	Done
Correlation ID	X-Correlation-Id header, request and response	Done
Error handling	GatewayErrorHandler.java (section 9)	Done
Postman collection	Save the 4 requests and export from Postman	Export needed
Architecture diagram	Section 4 of this document	Done
Documentation	This PDF	Done

12. Observations and Recommendations

Observation	Recommendation
/api/users returns every user's password field	Hide it with @JsonIgnore or return a DTO without the password
Passwords and JWT secret are written in application.properties	Move secrets to environment variables; never commit them to GitHub
notification-service and order-service both use port 8083	Change notification-service to another port (for example 8086)
Routes use fixed localhost addresses	In later epics use service discovery or Docker service names
Gateway forwards requests without authentication checks	Next step: validate JWT tokens at the Gateway

13. Conclusion

The API Gateway now acts as the single entry point for the user, product, order and payment services. It routes requests with path rewriting, logs every request with a unique correlation ID, forwards that ID to the services, and returns clear errors when a service is down. This completes JIRA Task 01 of EPIC 07.